

Bases de Datos

Recuperación de Fallas — Log Manager y Undo Logging

Lunes 25 de mayo

¿Dónde estamos?

- **Semana pasada:** transacciones y **ACID**.
 - **A**tomicity, **C**onsistency, **I**solation, **D**urability.
 - **2PL** y triggers → garantiza **I**solation.

Hoy y el miércoles: Atomicity y Durability — el otro par de ACID, asegurado por el **Log & Recovery Manager**.

El escenario

```
T1: UPDATE cuentas SET saldo = saldo - 100 WHERE cid = 1;  
T1: UPDATE cuentas SET saldo = saldo + 100 WHERE cid = 2;  
T1: COMMIT;
```

A nivel físico el DBMS hace:

```
READ(saldo1) → modificar en RAM → WRITE(saldo1 en disco)  
READ(saldo2) → modificar en RAM → WRITE(saldo2 en disco)  
COMMIT
```

¿Qué pasa si se corta la luz entre las dos escrituras?

→ La cuenta 1 quedó descontada. La cuenta 2 nunca recibió el dinero. **El banco perdió 100.**

Agenda

1. Tipos de fallas: qué cubre el Log & Recovery Manager.
2. **Log Manager**: registrar la actividad de las transacciones.
3. **Undo Logging**: deshacer transacciones incompletas.
4. **Checkpoints**: acotar el trabajo de recuperación.

¿Qué puede salir mal?

Tipos de fallas

Falla	Ejemplo	Solución
Datos erróneos	un cliente con RUT inválido	restricciones de integridad, data cleaning
Disco	sector defectuoso	RAID, replicación
Catástrofes	incendio del datacenter	backups geográficos
Sistema (crash)	corte de luz, pánico del SO	Log & Recovery Manager ← hoy

Supuestos del modelo

- La **RAM** es volátil: ante crash, lo que esté **sólo en RAM se pierde**.
- El **disco** es persistente: sobrevive al crash.
- El DBMS hace escrituras a disco en **páginas**. Mientras una página vive en el **buffer pool**, sus modificaciones **aún no son durables**.
- Un crash puede ocurrir en **cualquier instante**, incluso a mitad de una transacción o de una escritura.

Pregunta clave: ¿cómo reconstruimos el estado correcto de la BD después de un reinicio?

Log Manager

La idea

Antes (o después) de modificar el estado de la BD, **registramos qué hicimos** en un archivo secuencial llamado **log**.

- El log es **append-only**: se escribe linealmente.
- Es **mucho más barato** escribir secuencialmente al final de un archivo que actualizar páginas dispersas.
- Si la BD se pierde, podemos **reconstruirla** leyendo el log.

```
log:  <T1 start> <T1, A, 5, 7> <T1 commit> <T2 start> <T2, B, 3, 9> ...
```

Mecánica del log

- Las transacciones escriben en una **página de log en memoria**.
- Cuando la página se llena (o se hace **COMMIT**), se **flush** a disco.
- Todas las transacciones escriben **concurrentemente** en el mismo log secuencial.

T1 start
T1, A, ...
T2 start
T1 commit
T2, B, ...

página de log en RAM

flush al disco

log file
(disco)

Tipos de registros básicos

Registro	Significado
<START T>	la transacción T inició
<COMMIT T>	T confirmó su trabajo
<ABORT T>	T fue cancelada
<T, X, v_viejo, v_nuevo>	T modificó el objeto X (formato más completo)

El **contenido** del registro de update varía según la **estrategia de logging** (undo / redo / undo-redo). Hoy partimos con **Undo Logging** y asumimos un **Write-ahead log (WAL)**.

LSN — Log Sequence Number

Cada registro del log lleva un identificador único y monótono: el **LSN**.

```
LSN  log
1    <START T1>
2    <T1, A, 5>
3    <START T2>
4    <T2, B, 10>
5    <COMMIT T1>
...
```

- Permite **referirse** a un punto del log: "todo lo que esté antes del LSN 42 ya fue flusheado a disco".
- Cada página de datos puede almacenar el LSN del último cambio que la afectó → permite saber **qué necesita re-aplicarse**.
- En PostgreSQL es un offset literal en el WAL (visible en `pg_current_wal_lsn()`).

Undo Logging

Idea

Si la transacción no termina, **deshacemos** lo que alcanzó a hacer.

Para deshacer una escritura sobre X , necesitamos guardar su **valor antiguo**.

Registro	Significado
<START T>	T inició
<COMMIT T>	T confirmó
<ABORT T>	T fue cancelada
<T, X, t>	T modificó X; t era el valor antiguo

Las reglas de Undo

U1 (Write-Ahead Logging): si T modifica X, el log `<T, X, t>` debe estar en disco **antes** que la nueva versión de X.

U2a: antes de escribir `<COMMIT T>` en el log, **todas** las modificaciones de T deben estar **flusheadas a disco**.

U2b: `<COMMIT T>` debe estar **flusheado a disco** antes de reportar el commit al cliente.

- (1) escribir log `<T, X, t>` y flushearlos
- ↓
- (2) escribir X modificado en disco
- ↓
- (3) escribir `<COMMIT T>` en el buffer de log
- ↓
- (4) flushear el log con `<COMMIT T>` a disco
- ↓
- (5) recién ahora se le dice al cliente "OK, transacción confirmada"

¿Por qué funcionan estas reglas?

Imaginemos que el sistema falla en un punto cualquiera:

- **Antes de escribir** `<T, X, t>` : X tampoco está en disco modificado (regla U1) → no hay nada que deshacer.
- **Después del log pero antes de escribir X**: podemos deshacer X usando t.
- **Después de escribir X pero antes de** `<COMMIT T>` : el log dice que T no terminó → deshacemos.
- **Después de** `<COMMIT T>` : las modificaciones ya están en disco → no hay que deshacer.

El log nos dice **exactamente** qué T's quedaron a medio camino.

Algoritmo de recovery (Undo)

```
Completas =  $\emptyset$  (transacciones que ya sabemos resueltas)
para cada registro r del log, en orden inverso:
    si r = <COMMIT T> o <ABORT T>:
        Completas  $\leftarrow$  Completas  $\cup$  {T}
    si r = <T, X, t>:
        si T  $\notin$  Completas:
            escribir X := t en disco (deshacer)
    si r = <START T>:
        ignorar

para cada T sin COMMIT ni ABORT en el log:
    una vez completados todos sus undos,
    escribir <ABORT T> al final del log
```

Idempotencia. Si el sistema falla en medio de recovery, basta con volver a correrlo: reescribir `X := t` dos veces da el mismo resultado, y `<ABORT T>` se difiere hasta que el undo esté completo, de modo que el algoritmo nunca queda en un estado intermedio inconsistente.

Ejemplo

Log antes del crash:

```
<START T1>
<T1, A, 5>           ← A valía 5 antes; T1 lo modificó
<START T2>
<T2, B, 10>
<COMMIT T1>
<T2, C, 15>
x CRASH x
```

Recovery (de atrás hacia adelante):

- `<T2, C, 15>` : T2 no tiene COMMIT → **C := 15**.
- `<COMMIT T1>` : T1 está confirmada (marcar como completa).
- `<T2, B, 10>` : T2 incompleta → **B := 10**.
- `<T1, A, 5>` : T1 completa → ignorar.
- Escribir `<ABORT T2>` al final.

Problema: el log crece

- En un sistema con millones de transacciones, el log se vuelve **gigantesco**.
- Recovery tendría que recorrerlo entero. **Inaceptable**.

→ Necesitamos un mecanismo para decir *"hasta este punto del log, ya sabemos que todo está estable"*.

→ **Checkpoints**.

Checkpoints

Checkpoint quiescente (la versión naïve)

1. Dejar de aceptar nuevas transacciones.
2. Esperar a que terminen las activas.
3. Flush del log y de los datos sucios a disco.
4. Escribir <CKPT> en el log.
5. Reanudar transacciones.

Garantía: todo lo anterior a <CKPT> está estable en disco → recovery se detiene ahí.

Problema: hay que **pausar todo el sistema** mientras dura el checkpoint. Inviabile en producción OLTP.

El espíritu del checkpoint quiescente sí aparece en otros contextos: un `pg_dump` para backups lógicos o un *batch nocturno* en un data warehouse hacen algo parecido cuando la carga lo permite. Pero el DBMS principal no puede congelarse cada 5 minutos.

Checkpoint no-quiescente

Permitir que las transacciones sigan ejecutándose **durante** el checkpoint.

1. Escribir `<START CKPT (T1, ..., Tn)>` ← T1..Tn son las activas justo ahora
2. Permitir que T1..Tn (y nuevas) sigan trabajando
3. Esperar a que T1..Tn terminen (commit o abort)
4. Hacer flush de sus datos sucios a disco
5. Escribir `<END CKPT>`

`<START CKPT>` y `<END CKPT>` son **dos marcas** en el log que **encierran** un período de transición.

Recovery con checkpoint no-quiescente

Recorrer el log de atrás hacia adelante. Lo primero que se encuentre determina hasta dónde leer:

Caso 1: se encuentra primero `<END CKPT>` .

→ El checkpoint completó: todas las T's que estaban activas al `<START CKPT>` ya terminaron (commit o abort) y sus efectos están en disco.

→ Hay que deshacer **todas las transacciones sin COMMIT al momento del crash**, incluyendo cualquier transacción que **haya empezado después** del `<START CKPT>` y siga incompleta, pero no es necesario ver T1..Tn.

→ Retroceder en el log hasta el inicio de la más antigua entre ellas (va a ser después del último `<START CKPT>` .

Recovery con checkpoint no-quiescente

Recorrer el log de atrás hacia adelante. Lo primero que se encuentre determina hasta dónde leer:

Caso 2: se encuentra primero `<START CKPT (T1, ..., Tn)>` (sin un `<END CKPT>` posterior — el crash ocurrió a mitad de checkpoint).

→ No podemos confiar en que los datos de las T1..Tn estén en disco.

→ Retroceder al `<END CKPT>` **anterior** y deshacer todas las transacciones incompletas desde ese punto.

Ejemplo con checkpoint

```
<START T1>  
<T1, a, 5>  
<START T2>  
<T2, b, 10>  
<START CKPT (T1, T2)>  
<T2, c, 15>  
<START T3>  
<T1, d, 20>  
<COMMIT T1>  
<T3, e, 25>  
<COMMIT T2>  
<END CKPT>           ← checkpoint terminó OK  
x CRASH x
```

Recovery: encontramos `<END CKPT>` primero. Solo T3 puede estar incompleta. Undo de `<T3, e, 25>` → `e := 25`, luego escribir `<ABORT T3>` .

Mismo ejemplo, crash antes del END CKPT

```
<START T1>
<T1, a, 5>
<START T2>
<T2, b, 10>
<START CKPT (T1, T2)>
<T2, c, 15>
<START T3>
<T1, d, 20>
<COMMIT T1>
<T3, e, 25>
x CRASH x
```

← no llegó a escribir <END CKPT>

Recovery: encontramos `<START CKPT (T1, T2)>` sin su `<END CKPT>`. T2 y T3 quedaron incompletas. Tenemos que retroceder al `<START T2>` y deshacer todo lo de T2 y T3.

Tres preguntas que el checkpoint responde

- ¿Hasta dónde tengo que leer el log? Hasta el último `<END CKPT>` completo (más sus T pendientes).
- ¿Qué pasa si el sistema falla mientras estoy haciendo recovery? No importa: recovery es **idempotente** — volver a aplicarlo da el mismo resultado.
- ¿Cómo trunco el log? Puedo descartar todo lo anterior al último `<END CKPT>` cuyas T's están todas resueltas en disco.

Costo de Undo Logging

Para hacer `COMMIT` necesitamos:

1. Escribir el registro de update en log.
2. **Flush de las páginas modificadas a disco.**
3. Escribir `<COMMIT T>` en el log.
4. Flush del log.

→ **Las páginas modificadas se escriben *antes* del COMMIT** → cada commit espera I/O.

Para transacciones largas, esto es lento.

El miércoles: Redo Logging invierte esta regla — commit primero (en log), datos después. Y veremos por qué eso también tiene un costo.

Una nota: no todo el mundo usa WAL

El modelo WAL+páginas que estamos viendo es el de las BD **relacionales** clásicas. Es lo que usa Postgres, MySQL, Oracle. No es la única arquitectura posible.

La idea fundamental — *primero al log, después al estado* — sobrevive, pero hay muchos detalles técnicos que pueden funcionar distinto.

Resumen de hoy

- El **Log Manager** registra secuencialmente toda actividad de las transacciones.
- **Undo Logging** guarda **valores antiguos**; recovery deshace lo no confirmado.
- **Reglas U1/U2** garantizan que siempre podemos reconstruir un estado consistente.
- **Checkpoints** (idealmente no-quiescentes) acotan el trabajo de recovery.